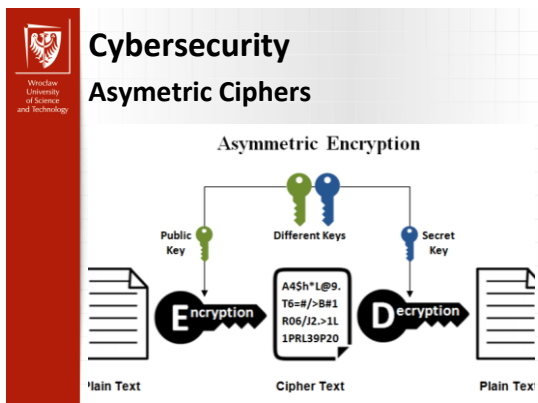




Cybersecurity

Agenda

- We briefly discuss the following
 - Merkle-Hellman knapsack
 - Diffie-Hellman key exchange
 - RSA
 - Rabin Cipher
 - Digital Signature

Cybersecurity

RSA

- Invented by Cocks, independently, by Rivest, Shamir, Adleman (MIT)
- Let p and q be two large prime numbers
- Let $N = pq$ be the **modulus**
- Choose e relatively prime to $(p-1)(q-1)$
- Find d so that $ed = 1 \pmod{(p-1)(q-1)}$
- Public key** is (N, e)
- Private key** is d

Cybersecurity

RSA

- To encrypt M compute: $C = M^e \pmod{N}$
- To decrypt C compute: $M = C^d \pmod{N}$
- Recall that e and N are public
- If attacker can factor N** , can use e to easily find d since $ed = 1 \pmod{(p-1)(q-1)}$
- Factoring the modulus breaks RSA!
- It is not known whether factoring is the only way to break RSA

Cybersecurity

RSA

- Given $C = M^e \pmod{N}$ we must show $M = C^d \pmod{N} = M^{ed} \pmod{N}$
- We use **Euler's Theorem**:
If x is relatively prime to n then $x^{\phi(n)} = 1 \pmod{n}$
 - Fact: $ed = 1 \pmod{(p-1)(q-1)}$
 - Fact: $ed = k(p-1)(q-1) + 1$
 - Fact: $\phi(N) = (p-1)(q-1)$
 - Fact: $ed - 1 = k(p-1)(q-1) = k\phi(N)$

$$M^{ed} = M^{(ed-1)+1} = M \cdot M^{ed-1} = M \cdot M^{k\phi(N)} = M \cdot (M^{\phi(N)})^k = M \cdot 1^k = M \pmod{N}$$

Cybersecurity

RSA

- Example of RSA
 - Select "large" primes $p = 11$, $q = 3$
 - Then $N = pq = 33$ and $(p-1)(q-1) = 20$
 - Choose $e = 3$ (relatively prime to 20)
 - Find d such that $ed = 1 \pmod{20}$, we find that $d = 7$ works
- Public key**: $(N, e) = (33, 3)$
- Private key**: $d = 7$



Cybersecurity

RSA

- **Public key:** $(N, e) = (33, 3)$
- **Private key:** $d = 7$
- Suppose message $M = 8$
- Ciphertext C is computed as

$$C = M^e \pmod{N} = 8^3 = 512 = 17 \pmod{33}$$
- Decrypt C to recover message:

$$M = C^d \pmod{N} = 17^7 = 410,338,673$$

$$12,434,505 * 33 + 8 = 8 \pmod{33}$$

=



Cybersecurity

RSA

- RSA is the “gold standard” in public key crypto
- Provides encryption and signatures
- Has stood the test of time
 - unchanged since its invention



Cybersecurity

Rabin Cipher

- Choose $N = pq$, where p and q prime
- Assume $p \equiv 3 \pmod{4}$ and $q \equiv 3 \pmod{4}$
- Public key: N
- Private key: (p, q)
- Encrypt: $C = M^2 \pmod{N}$
- Decrypt: Given p and q , we must find the square root of C , modulo N



Cybersecurity

RSA

- Factorization of big N

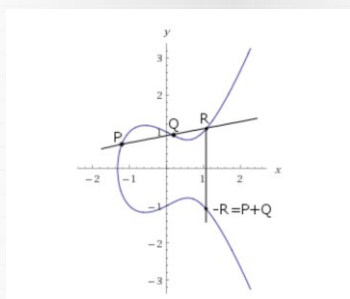
Bit-length of N	Year
400	1993
478	1994
515	1999
768	2009
795	2019

for long-term security



Cybersecurity

Elliptic Curve Cryptography (ECC)



Cybersecurity

Elliptic Curve Cryptography (ECC)

- The ECC addition operation is equivalent to modular multiplication in RSA
- Multiple addition is equivalent to modular exponentiation

To create a cryptographic system using elliptic curves, we need to find a “difficult problem” corresponding to the factorization of the product of two prime numbers or assuming a discrete logarithm

- $Q = kP$, where Q, P belong to the first curve
- “Easy” to calculate Q , taking into account k and P
- But “hard” to find k for Q and P
- Known as a problem with the logarithm of the elliptic curve

ECC Encryption/Decryption

- Several approaches using elliptic curves were analyzed
- You must first code any m message as a point on the elliptic curve P_m
- Select the appropriate curve and G point as in Diffie-Hellman
- Each user selects the private key n_A and generates a public key $P_A = n_A * G$
- To encrypt and send a message P_m to B , A selects a random positive integer k and creates encrypted text C_m consisting of a pair of points:

$$C_m = \{kG, P_m + kP_B\}$$

- To decrypt encrypted text, B multiplies the first point in pair by the secret key B and subtracts the result from the second point:

$$P_m + kP_B - n_B(kG) = P_m + k(n_B G) - n_B(kG) = P_m$$

Elliptic Curve Cryptography Security

- It depends on the difficulty of the problem with the logarithm of the elliptic curve
- The fastest known technique is the "Pollard rho method"
- Compared to factoring, it can use much smaller key sizes than RSA
- For equivalent key lengths, the calculations are approximately equivalent
- Therefore, ECC offers significant computational benefits for similar security

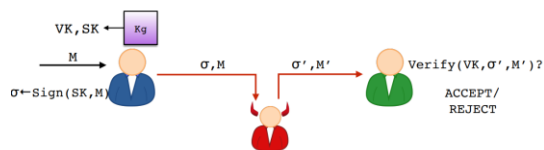
- Comparable key sizes in terms of computational effort in cryptanalysis (NIST SP-800-57)

Symmetric key algorithms	Diffie-Hellman, Digital Signature Algorithm	RSA (size of n in bits)	ECC (modulus size in bits)
80	$L = 1024$ $N = 160$	1024	160–223
112	$L = 2048$ $N = 224$	2048	224–255
128	$L = 3072$ $N = 256$	3072	256–383
192	$L = 7680$ $N = 384$	7680	384–511
256	$L = 15,360$ $N = 512$	15,360	512+

Note: L = size of public key, N = size of private key

Cybersecurity

Digital Signature



Cybersecurity

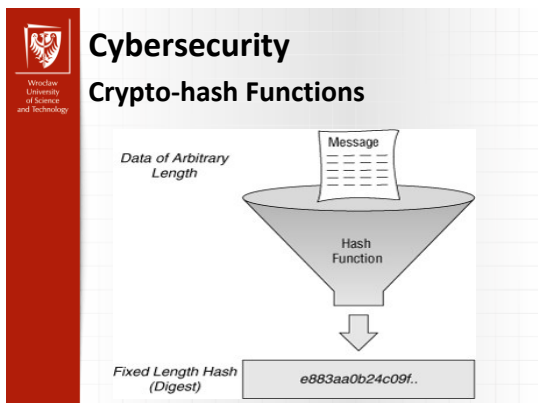
Summary

- Advantages of asymmetric-key crypto:**
 - Only the private key needs to remain secret
 - The administration of keys on a network requires the presence of only a functionally trusted (honest and fair) TTP.
 - Depending on the mode of usage, the public and private key pairs may be used for long periods of time (upper bound: Moore's Law).
 - In large networks, n keys are required instead of $(n)*(n-1)/2$
- Disadvantages of asymmetric-key crypto:**
 - Throughput rates are typically very slow (all known algorithms)
 - Key sizes are typically much larger (1024 - 4096 bits)
 - Security is based upon the presumed difficulty of a small set of number-theoretic problems and all known are subject to short-cut attacks (e.g. knowing the prime factorisation of n)

Cybersecurity

Summary

- Symmetric and asymmetric crypto are complementary
- Public key crypto can be used to establish a key for fast symmetric crypto (e.g. a session key)
- Alice and Bob take advantage of the long term benefits of public key crypto and publish their public keys in a directory
- Public key crypto is good for key management and signatures
- Private key crypto (symmetric) is good for encryption and some data integrity applications



Cybersecurity

Agenda

- Non-crypto hash functions
- Crypto hash function – properties
- Digital Signatures
- Signature Birthday Attack

Cybersecurity

What are hash functions?

- Just a method of compressing strings
 - E.g., $H : \{0,1\}^* \rightarrow \{0,1\}^{160}$
 - Input is called “message”, output is “digest”
- Why would you want to do this?
 - Short, fixed-size better than long, variable-size
 - True also for non-crypto hash functions
 - Digest can be added for redundancy
 - Digest hides possible structure in message

Cybersecurity

Non-crypto Hash (1)

- Data $X = (X_0, X_1, X_2, \dots, X_{n-1})$, each X_i is a byte
- Let $\text{hash}(X) = X_0 + X_1 + X_2 + \dots + X_{n-1}$
- Is this secure?
- Example: $X = (10101010, 00001111)$
- Hash is 10111001
- But so is hash of $Y = (00001111, 10101010)$
- Easy to find collisions, so **not** secure...

Cybersecurity

Non-crypto Hash (2)

- Data $X = (X_0, X_1, X_2, \dots, X_{n-1})$
- Suppose hash is
 - $h(X) = nX_0 + (n-1)X_1 + (n-2)X_2 + \dots + 1 \cdot X_{n-1}$
- Is this hash secure? At least
 - $h(10101010, 00001111) \neq h(00001111, 10101010)$
- But hash of $(00000001, 00001111)$ is same as hash of $(00000000, 00010001)$
- Not secure, but it is used in the (non-crypto) application **rsync**

Cybersecurity

Non-crypto Hash (3)

- Cyclic Redundancy Check (CRC)
- Essentially, CRC is the remainder in a long division calculation
- Good for detecting burst **errors**
- Easy to construct collisions
- CRC sometimes mistakenly used in crypto applications (WEP)

